SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I ____S Hariram (1965260018)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

QUESTION 1: HOSPITAL DOCTOR SHIFT ASSIGNMENT

1. Problem Formulation

Variables: D1, D2, D3 (Doctors)

Domains: For each Di, Domain(Di) = {Morning, Afternoon, Night}

Shift Hierarchy: Morning (1) < Afternoon (2) < Night (3)

Constraints:

1. D1 ≠ Night
2. D2 < D3 (The shift of D2 must strictly precede the shift of D3)
3. All Different(D1, D2, D3) (Only one doctor per shift)
4. D3 ≠ Morning
5. All doctors must be assigned exactly one shift.

2. Intermediate Backtracking Steps and Constraint Checks

Variable Assignment Order: D1 → D2 → D3

Domain Value Test Order: Morning → Afternoon → Night

Step 1: Assign D1 = Morning

- Check Constraint (i): D1 ≠ Night → Morning ≠ Night (Satisfied).
- Current Assignment: {D1: Morning}.

Step 2: Try D2 = Morning

- Check Constraint (iii): D2 ≠ D1 → Morning = Morning (Violated).
- Action: Backtrack.

Step 3: Try D2 = Afternoon

- Check Constraint (iii): D2 ≠ D1 → Afternoon ≠ Morning (Satisfied).
- Current Assignment: {D1: Morning, D2: Afternoon}.

Step 4: Try D3 = Morning

- Check Constraint (iv): D3 ≠ Morning (Violated).
- Check Constraint (iii): D3 ≠ D1 (Violated).
- Action: Backtrack.

Step 5: Try D3 = Afternoon

- Check Constraint (iii): D3 ≠ D2 → Afternoon = Afternoon (Violated).
- Action: Backtrack.

Step 6: Try D3 = Night

- Check Constraint (iii): Night ≠ Morning and Night ≠ Afternoon (Satisfied).
- Check Constraint (iv): D3 ≠ Morning → Night ≠ Morning (Satisfied).
- Check Constraint (ii): D2 < D3 → Afternoon < Night (2 < 3) (Satisfied).
- Action: Valid complete assignment reached.

3. Final Valid Schedule

Shift	Doctor Assigned
Morning (Shift 1)	Doctor D1
Afternoon (Shift 2)	Doctor D2
Night (Shift 3)	Doctor D3

Final Assignment: {D1 = Morning, D2 = Afternoon, D3 = Night}

Conclusion: The backtracking search successfully finds a valid assignment that satisfies all the given constraints. Whenever a constraint is violated, the algorithm backtracks and tries the next possible value until a valid complete assignment is found.

QUESTION 2: 5×5 GRID PATHFINDING USING BFS WITH PRIORITY QUEUE

1. Problem Formulation

- **Grid:** 5 × 5 grid with coordinates (r, c), where r, c = 0, 1, 2, 3, 4.
- **Start:** (0, 0)
- **Goal:** (4, 4)
- **Obstacles:** (1, 1), (1, 2), (2, 3), (3, 1)
- **Allowed Moves:** Up, Down, Left and Right.
- **Step Cost:** 1 for each move.
- **Heuristic:** Manhattan Distance
  $h(r, c) = |r - 4| + |c - 4|$

The search uses a priority queue that selects the node with the lowest path cost g(n). If two nodes have the same path cost, the node with the smaller heuristic value h(n) is selected.

2. Step-by-Step Search Process

Step	Expanded Node	g(n)	h(n)	New Neighbors Added	Priority Queue (Front to Back)
Init	—	—	—	—	(0,0)
1	(0,0)	0	8	(1,0), (0,1)	(1,0), (0,1)
2	(1,0)	1	7	(2,0)	(0,1), (2,0)
3	(0,1)	1	7	(0,2)	(2,0), (0,2)
4	(2,0)	2	6	(3,0), (2,1)	(0,2), (3,0), (2,1)
5	(0,2)	2	6	(0,3)	(3,0), (2,1), (0,3)
6	(3,0)	3	5	(4,0)	(2,1), (0,3), (4,0)
7	(2,1)	3	5	(2,2)	(0,3), (4,0), (2,2)
8	(0,3)	3	5	(0,4), (1,3)	(4,0), (2,2), (0,4), (1,3)
9	(4,0)	4	4	(4,1)	(2,2), (0,4), (1,3), (4,1)
10	(2,2)	4	4	(3,2)	(0,4), (1,3), (4,1), (3,2)
11	(0,4)	4	4	(1,4)	(1,3), (4,1), (3,2), (1,4)
12	(1,3)	4	4	None	(4,1), (3,2), (1,4)
13	(4,1)	5	3	(4,2)	(3,2), (1,4), (4,2)
14	(3,2)	5	3	(3,3)	(1,4), (4,2), (3,3)

15	(1,4)	5	3	(2,4)	(4,2), (3,3), (2,4)
16	(4,2)	6	2	(4,3)	(3,3), (2,4), (4,3)
17	(3,3)	6	2	(3,4)	(2,4), (4,3), (3,4)
18	(2,4)	6	2	None	(4,3), (3,4)
19	(4,3)	7	1	(4,4)	(3,4), (4,4)
20	(3,4)	7	1	None	(4,4)
21	(4,4)	8	0	Goal reached	Empty

3. Optimal Path and Cost

Optimal Path:

(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Total Number of Moves: 8

Total Path Cost: 8 × 1 = 8

Result

The search reaches the goal at (4,4) without passing through any obstacle. The shortest path contains 8 moves, and the total cost is 8.

QUESTION 3: AUTONOMOUS DISASTER RESCUE ROBOT NAVIGATION

Task (a): Constraint Analysis and Impact on Decision-Making

1. **Movement Constraints:** The robot can move only in four directions: Up, Down, Left and Right. It cannot move diagonally.

2. **Unknown Obstacles:** The robot does not know the locations of all obstacles in advance. It must explore the area and find a safe path while moving.
3. **Limited Sensing:** The robot can detect obstacles and hazards only in the neighboring rooms. Therefore, it must update its map whenever it receives new information.
4. **Different Movement Costs:** Moving through a normal room costs 1, while moving through a risky room costs 3. The robot should prefer a safe detour if it costs less than passing through a risky room.
5. **Continuous Updates:** The robot must update its knowledge of the environment after every move. This helps it avoid dead ends and choose a better path when new obstacles are discovered.

Task (b): Solution Approach and Search Strategy

An *Online Search Agent using LRTA or D* Lite** is suitable for this problem because the robot does not have complete information about the environment before starting its journey.

Step 1: Initialize the Map

Create a grid to store information about the rooms. Initially, unknown rooms are assumed to be passable, with a movement cost of 1.

Step 2: Detect Obstacles and Hazards

The robot uses its sensors to check nearby rooms. Blocked rooms are assigned an infinite cost, normal rooms have a cost of 1, and risky rooms have a cost of 3.

Step 3: Evaluate Neighboring Rooms

For each available neighboring room, calculate the estimated cost using:

$f(v) = c(u, v) + H(v)$

Here, $c(u, v)$ is the cost of moving to the neighboring room, and $H(v)$ is the estimated cost from that room to the survivor.

Step 4: Update the Heuristic Value

The robot updates its estimate of the current room using:

$H(u) = \max(H(u), \min[c(u, v) + H(v)])$

The minimum is calculated over the available neighboring rooms. This helps the robot improve its estimates as it explores the environment.

Step 5: Move and Repeat

The robot selects a suitable neighboring room with the lowest estimated cost, moves to it, and checks the surroundings again. This process continues until the survivor is reached.

Task (c): Core AI Concepts

1. Intelligent Agent Formulation (PEAS)

- **Performance Measure:** Reach the survivor safely, minimize travel cost and time, and avoid collisions.
- **Environment:** A disaster-affected building containing blocked paths, normal rooms and risky areas.
- **Actuators:** Movement controls for Up, Down, Left and Right.
- **Sensors:** Proximity sensors to detect obstacles, hazard sensors to identify risky rooms, and localization sensors to track the robot's position.

2. Rationality

A rational agent selects actions that help achieve its goal while minimizing the expected cost. Since the robot cannot see the entire environment, it makes decisions using the information available from its sensors and previous movements. It also updates its knowledge when new obstacles or hazards are discovered.

3. Environment Classification

- **Partially Observable:** The robot can observe only the neighboring rooms.
- **Deterministic:** Each movement produces a predictable result.
- **Sequential:** Every movement affects the robot's future position and available paths.
- **Static:** The building structure is assumed to remain unchanged during navigation, although the robot's knowledge changes as it explores.
- **Discrete:** The environment consists of separate grid cells and a limited set of actions.
- **Single-Agent:** Only the rescue robot makes decisions in the environment.

Task (d): Justification of the Chosen Approach

1. **Suitable for Unknown Environments:** Traditional offline search methods require a known map. Online search allows the robot to discover obstacles while moving and adjust its route accordingly.
2. **Efficient Replanning:** Incremental methods such as D* Lite can reuse information from previous searches instead of calculating the entire route again whenever the map changes.
3. **Considers Movement Costs:** The robot considers the higher cost of risky rooms and can choose a longer but safer route when it has a lower total cost.
4. **Adapts to New Information:** The robot updates its map and path whenever it detects new obstacles or hazards. This makes it more suitable for disaster rescue operations.

Conclusion: An online search agent with incremental replanning is a suitable choice for autonomous disaster rescue. It allows the robot to explore an unknown environment, avoid obstacles, consider risky areas, and find a low-cost path to the survivor.